



Università di Genova

High Performance Computing Report Mandelbrot Algorithm

Andrea Cattarinich 5137057

November 29, 2025

Contents

1	Architectures	3
1.1	Personal Workstation	3
1.2	University Workstation	3
1.3	Google Colab	4
2	Mandelbrot Set	5
2.1	Definition	5
2.1.1	Point that belongs to the Mandelbrot set	5
2.1.2	Point that does NOT belong to the Mandelbrot set	5
2.2	Images	5
3	Hotspot Analysis	6
3.1	Optimization Problems	6
3.1.1	Solution and Results	6
3.2	Hotspot Identification	7
3.3	Conclusions	8
4	OpenMP	9
4.1	Parallelization over the outer loop	9
4.2	Parallelization over the inner loop with reduction	9
4.3	Results	10
5	CUDA	12
5.1	Sequential section	12
5.2	CUDA	12
5.2.1	Code	12
5.2.2	Compilation	13

5.2.3	Execution	14
5.2.4	Results	14
6	Conclusions	16

1 Architectures

For this project I have used three workstation, my personal one, the workstation provided by the University and Google Colab.

1.1 Personal Workstation

HP Envy 17-ae103nl

CPU

Model	Intel Core i7-6500U @ 2.50 GHz
Architecture	x86_64
Physical cores	2
Logical threads	4 (Hyper-Threading)

GPU

Model	NVIDIA GeForce GTX 950M
CUDA Compute Capability	5.0
Driver version	572.16
CUDA Version	12.8

Software and Setup

Visual Studio	Visual Studio Community 2022 (C++ Desktop workload)
CUDA Toolkit	Installed manually
WSL2	Enabled
NVIDIA Drivers	Version 572.16 (for WSL2)

1.2 University Workstation

Room 210

CPU

Model	Intel Core i9-12900K (12th Gen)
Architecture	x86_64
CPU(s)	24
Threads per core	2
Cores per socket	16
Socket(s)	1
Max frequency	4.02 GHz
Profiling tools	Intel VTune Profiler, Intel Advisor

To use the Intel oneAPI tools, the environment was loaded with:

```
source /opt/intel/oneapi/setvars.sh
```

1.3 Google Colab

CPU

Model	Intel(R) Xeon(R) CPU @ 2.20GHz
Architecture	x86_64
Physical cores	1
Logical threads	2 (Hyper-Threading)
Socket(s)	1

GPU

Model	NVIDIA Tesla T4
CUDA Compute Capability	7.5
Driver version	550.54.15
CUDA Version	12.8

2 Mandelbrot Set

2.1 Definition

The Mandelbrot fractal is defined as a set of points in the complex plane. Each point corresponds to a complex number c .

For every c , we consider the iterative sequence:

$$z_0 = 0, \quad z_{n+1} = z_n^2 + c$$

We then observe the behavior of z_n as $n \rightarrow \infty$.

2.1.1 Point that belongs to the Mandelbrot set

Take $c = -1$. Then the sequence is

$$z_0 = 0, \quad z_1 = 0^2 + (-1) = -1, \quad z_2 = (-1)^2 + (-1) = 0, \quad z_3 = 0^2 + (-1) = -1, \dots$$

The results remain bounded, so $c = -1$ belongs to the Mandelbrot set.

2.1.2 Point that does NOT belong to the Mandelbrot set

Take $c = 1$. Then the sequence is

$$z_0 = 0, \quad z_1 = 0^2 + 1 = 1, \quad z_2 = 1^2 + 1 = 2, \quad z_3 = 2^2 + 1 = 5, \dots$$

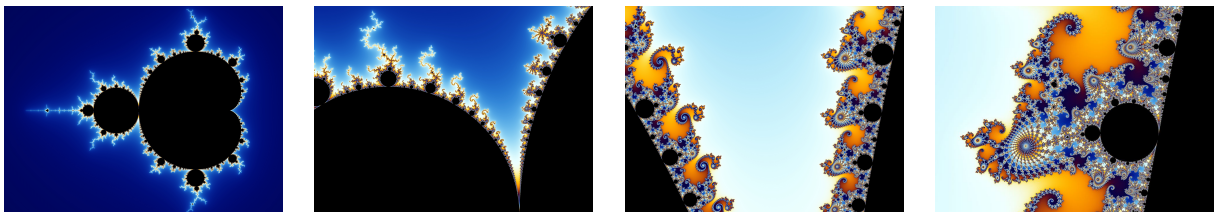
The values grow up to infinite, so $c = 1$ does not belong to the Mandelbrot set.

In our specific case, the escape condition is defined as:

$$|z_i| > 2$$

2.2 Images

This process gives rise to the famously intricate structure of the Mandelbrot fractal. Here a simple explanation [1] or here a more complex one [2] and here an online demo [3].



The color represents how quickly the values of z_n grow:

Black - the sequence remains bounded;

Other colors - the sequence escapes to infinity, with the color indicating how fast it diverges.

3 Hotspot Analysis

3.1 Optimization Problems

Questa istruzione non e' vettorizzabile con MSVC, e per tanto, l'esecuzione risulta essere poco performante.

Mentre con Linux G++, l'operazione sembra ottimizzare `std::complex`.

```
1 z = pow(z, 2) + c;
```

L'operazione `std::complex` utilizza l'overloading, in particolare `operator+` e `operator*`, creando un overhead e un conseguente carico sulla CPU. Inoltre, l'operazione `abs(z)` calcola la radice quadrata ogni volta.

Per rimuovere questa dipendenza dovuta dai numeri complessi, ho realizzato un miglioramento nel codice che mi consente di usare solamente numeri reali. Infatti, sfruttando la seguente relazione

$$z^2 = (a, b)^2 = (a^2 - b^2, 2ab)$$

Ho implementato:

$$z^2 = (\text{Re}(z), \text{Im}(z))^2 = (\text{Re}(z)^2 - \text{Im}(z)^2, 2 \text{Re}(z) \text{Im}(z))$$

Come segue:

```
1 double zRe = 0, zIm = 0;
2 for (int i = 1; i <= ITERATIONS; i++)
3 {
4     double oldRe = zRe;
5     double zRe2 = zRe*zRe; // Re(z)^2
6     double zIm2 = zIm*zIm; // Im(z)^2
7
8     zRe = (zRe2 - zIm2) + cRe;
9     zIm = 2*oldRe*zIm + cIm;
10
11     if(zRe2 + zIm2 >= 4)
12     {
13         image[pos] = i;
14         break;
15     }
16 }
```

3.1.1 Solution and Results

I risultati, come ci aspettavamo sono più performanti.

MSVC

```
cl /O2 /arch:AVX2 /fp:fast /EHsc src\*.cpp /Fe:build\*.exe
```

WSL

```
g++ -O2 -o build/* src/*.cpp
```

	mandelbrot	opt1	opt2
WSL	96s	30s (x3.2)	36s (x2.7)
Windows	-	40s	36 s

Utilizzeremo questo codice come baseline per lo sviluppo della restante parte del progetto.

3.2 Hotspot Identification

The measurements were performed using Intel VTune Profiler and the analysis was performed using Intel Advisor.

Function Stack	Total	Percentage (%)
Total	69.97	100.00
_start	69.97	100.00
__libc_start_main_impl	69.97	100.00
– main	69.97	100.00
— std::abs<double>	41.2121	58.90
— std::operator+<double>	13.6361	19.49
— std::pow<double>	10.712	15.31
— std::complex<double>::complex	0.0199997	0.03

The two main hotspots are the following instructions which takes the most fraction of the program execution.

Line 48

```
1 z = pow(z, 2) + c;
```

Line 51

```
1 if (abs(z) >= 2)
```

Source Line ▲	Source	🔥 CPU Time: Total	CPU Time: Self
31			
32	const auto start = chrono::steady_clock::now();		
33	for (int pos = 0; pos < HEIGHT * WIDTH; pos++)	0.0%	0.008s
34	{		
35	image[pos] = 0;	0.0%	0.012s
36			
37	const int row = pos / WIDTH;	0.0%	0.008s
38	const int col = pos % WIDTH;	0.0%	0.012s
39	const complex<double> c(col * STEP + MIN_X, row * STEP + MIN_Y);	0.0%	0.012s
40			
41	// z = z^2 + c		
42	complex<double> z(0, 0);		
43	for (int i = 1; i <= ITERATIONS; i++)		
44	{		
45	z = pow(z, 2) + c;	39.9%	3.570s
46			
47	// If it is convergent		
48	if (abs(z) >= 2)	59.9%	0.732s
49	{		
50	image[pos] = i;		
51	break;		
52	}		
53	}	0.0%	0.024s
54	}	0.0%	0.012s

3.3 Conclusions

The main hotspot is the DFT() function and its inverse IDFT(), which together account for over 99,05% of the total execution time. All other functions have negligible impact. This indicates that **parallelization and vectorization efforts should focus on the double loop inside DFT()**.

The main hotspot is the loop contained in lines 72-80 (source: src/omp_homework.c). The dominant operations contributing to the algorithm's computational intensity are the `sin()` and `cos()` trigonometric functions, which account for the majority of the execution time, as shown in the following code.

Source Line ▲	Source	🔥 CPU Time: Total ⓘ	CPU Time: Self ⓘ
31			
32	const auto start = chrono::steady_clock::now();		
33	for (int pos = 0; pos < HEIGHT * WIDTH; pos++)	0.0%	0.008s
34	{		
35	image[pos] = 0;	0.0%	0.012s
36			
37	const int row = pos / WIDTH;	0.0%	0.008s
38	const int col = pos % WIDTH;	0.0%	0.012s
39	const complex<double> c(col * STEP + MIN_X, row * STEP + MIN_Y);	0.0%	0.012s
40			
41	// z = z^2 + c		
42	complex<double> z(0, 0);		
43	for (int i = 1; i <= ITERATIONS; i++)		
44	{		
45	z = pow(z, 2) + c;	39.9%	3.570s
46			
47	// If it is convergent		
48	if (abs(z) >= 2)	59.9%	0.732s
49	{		
50	image[pos] = i;		
51	break;		
52	}		
53	}	0.0%	0.024s
54	}	0.0%	0.012s

4 OpenMP

4.1 Parallelization over the outer loop

In the first approach (`src/omp1.c`), the outer loop over the frequency index k is parallelized.

```
1 #pragma omp parallel for private(n)
2 for (k = 0; k < N; k++) {
3     double Xr_temp = 0.0;
4     double Xi_temp = 0.0;
5     for (n = 0; n < N; n++) {
6         Xr_temp += xr[n] * cos(n * k * PI2 / N) + idft * xi[n] *
7             sin(n * k * PI2 / N);
8         Xi_temp += -idft * xr[n] * sin(n * k * PI2 / N) + xi[n] *
9             cos(n * k * PI2 / N);
10    }
11    Xr_o[k] = Xr_temp;
12    Xi_o[k] = Xi_temp;
13 }
```

Here, each thread computes a subset of the output frequencies independently. The temporary variables `Xr_temp` and `Xi_temp` are private to each thread, ensuring there is no race condition when updating the output arrays.

4.2 Parallelization over the inner loop with reduction

In the second approach, the inner loop over the time index n is parallelized using a reduction clause:

```
1 for (k = 0; k < N; k++) {
2     double Xr_temp = 0.0;
3     double Xi_temp = 0.0;
4     #pragma omp parallel for reduction(+:Xr_temp, Xi_temp)
5     for (n = 0; n < N; n++) {
6         Xr_temp += xr[n] * cos(n * k * PI2 / N) + idft * xi[n] *
7             sin(n * k * PI2 / N);
8         Xi_temp += -idft * xr[n] * sin(n * k * PI2 / N) + xi[n] *
9             cos(n * k * PI2 / N);
10    }
11    Xr_o[k] = Xr_temp;
12    Xi_o[k] = Xi_temp;
13 }
```

Here, the computation of each output frequency $X[k]$ is parallelized across the time samples n . The `reduction` clause ensures that each thread accumulates contributions to `Xr_temp` and `Xi_temp` safely, avoiding race conditions. This method can provide higher parallel efficiency for large N , but incurs some overhead from the reduction operation, as shown in the following figures.

4.3 Results

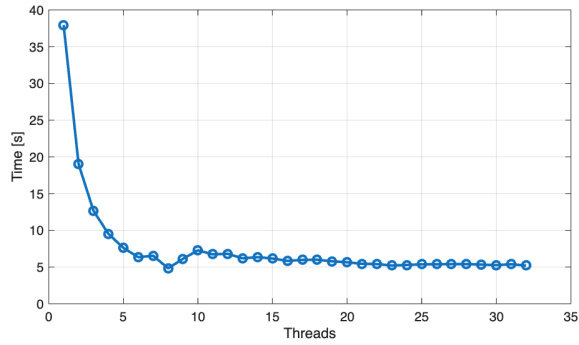
Parallelized for loop with OpenMP using `private(n)` to make each thread independent.

```

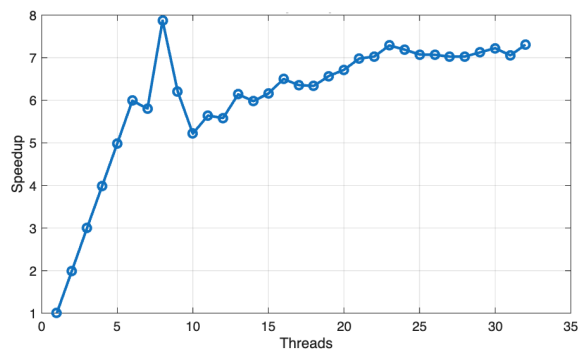
1 #pragma omp parallel for private(n)
2 for (k=0 ; k<N ; k++)
3 {
4     double Xr_temp = 0.0;
5     double Xi_temp = 0.0;
6
7     for (n=0 ; n<N ; n++) {
8         Xr_temp += ...
9         Xi_temp += ...
10    }
11
12    Xr_o[k] = Xr_temp;
13    Xi_o[k] = Xi_temp;
14 }

```

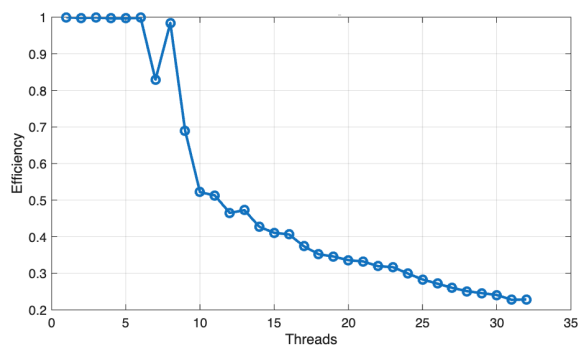
Execution time



Speedup



Efficiency



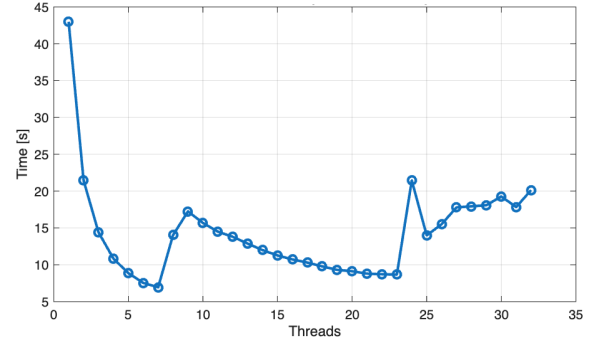
Parallelized for loop with OpenMP using `reduction` to combine the results from all threads.

```

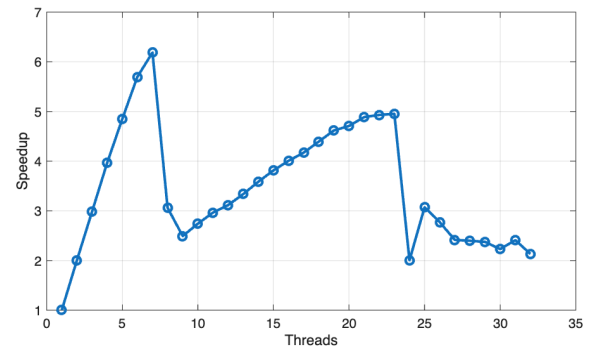
1 int k, n;
2 for (k=0 ; k<N ; k++)
3 {
4     double Xr_temp = 0.0;
5     double Xi_temp = 0.0;
6
7 #pragma omp parallel for reduction(+:
8     Xr_temp, Xi_temp)
9     for (n=0 ; n<N ; n++) {
10         Xr_temp += ...
11         Xi_temp += ...
12    }
13
14    Xr_o[k] = Xr_temp;
15    Xi_o[k] = Xi_temp;
16 }

```

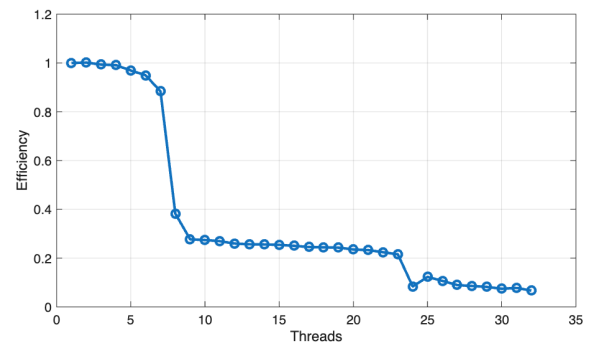
Execution time



Speedup



Efficiency



Precomputation of Trigonometric Functions: in an additional attempt to optimize the DFT computation, I modified the code as in the source `src/omp3.c`. The idea was to reduce the repeated evaluation of trigonometric functions by computing the sine and cosine values once per iteration:

```
1 #pragma omp parallel for private(n)
2 for (k=0; k<N; k++) {
3     double Xr_temp = 0.0;
4     double Xi_temp = 0.0;
5
6 #pragma omp simd reduction(+:Xr_temp, Xi_temp)
7     for (n=0; n<N; n++) {
8         double angle = n * k * PI2 / N;
9         double c = cos(angle);
10        double s = sin(angle);
11
12        Xr_temp += xr[n]*c + idft*xi[n]*s;
13        Xi_temp += -idft*xr[n]*s + xi[n]*c;
14    }
15
16    Xr_o[k] = Xr_temp;
17    Xi_o[k] = Xi_temp;
18 }
```

The motivation behind this modification was that, theoretically, computing the trigonometric functions only once per iteration should reduce the overall workload and improve performance.

However, the experiments did not yield significant improvements. Performance remained the same, or in some cases slightly worse. This is likely due to the compiler automatically applying this optimization by default.

5 CUDA

5.1 Sequential section

Ho deciso inizialmente di eseguire i test sul mio PC personale, piuttosto che sulla workstation dell'università o su Google Colab, per evitare limitazioni dovute a sessioni temporanee e risorse limitate. In questo modo ho avuto maggiore controllo sul processo di compilazione e sull'esecuzione dei test.

Questa parte del report fa riferimento solo all'analisi eseguita tramite il mio PC personale. I parametri della simulazione utilizzati sono:

- $nstep = 200$
- $ni = 15000$
- $nj = 15000$
- $size = ni \times nj = 900\,000\,000$

Ho compilato il codice sequenziale presente in `src/cuda.c` using the following options:

```
gcc -O3 src/cuda.c -o build/cuda
```

Il tempo di esecuzione migliore ottenuto (37s) costituirà il riferimento sequenziale per il calcolo dello speedup nella versione GPU.

5.2 CUDA

5.2.1 Code

La parallelizzazione è stata realizzata utilizzando CUDA per eseguire il calcolo dei passi temporali su GPU. Il kernel principale è `step_kernel_mod`, che replica il comportamento della funzione sequenziale `step_kernel_ref`.

Il codice si può trovare in `src/cuda.cu`

Struttura del kernel:

- Ogni thread gestisce un punto della griglia 2D.
- Gli indici globali i e j vengono calcolati a partire dai `threadIdx` e `blockIdx`.

Gestione della memoria:

- Allocazione della memoria host e device.
- Copia dei dati iniziali dal host alla device.
- Copia dei risultati dalla device al host al termine della simulazione.

Controllo errori:

- Dopo ogni chiamata al kernel viene eseguito `cudaGetLastError()`.
- Viene calcolato l'errore massimo rispetto al riferimento CPU per verificare la correttezza.

```
1 __global__ void step_kernel_mod(int ni, int nj, float fact, float
  * temp_in, float* temp_out)
2 {
3   int i00, im10, ip10, i0m1, i0p1;
4   float d2tdx2, d2tdy2;
5
6   int i = threadIdx.x + blockIdx.x * blockDim.x;
7   int j = threadIdx.y + blockIdx.y * blockDim.y;
8
9   if(i > 0 && i < ni-1 && j > 0 && j < nj-1){
10    // find indices into linear memory
11    // for central point and neighbours
12    i00 = I2D(ni, i, j);
13    im10 = I2D(ni, i-1, j);
14    ip10 = I2D(ni, i+1, j);
15    i0m1 = I2D(ni, i, j-1);
16    i0p1 = I2D(ni, i, j+1);
17
18    // evaluate derivatives
19    d2tdx2 = temp_in[im10]-2*temp_in[i00]+temp_in[ip10];
20    d2tdy2 = temp_in[i0m1]-2*temp_in[i00]+temp_in[i0p1];
21
22    // update temperatures
23    temp_out[i00] = temp_in[i00]+fact*(d2tdx2 + d2tdy2);
24  }
25 }
```

5.2.2 Compilation

La compilazione CUDA è stata effettuata con il comando:

```
nvcc -O3 -arch=sm_50 cuda.cu -o cuda.exe -DDEBUG=0
```

Significato dei flag:

- `-O3`: ottimizzazione massima.
- `-arch=sm_50`: architettura target della GPU.
- `-DDEBUG=0`: disabilita il debug per migliorare le performance.

5.2.3 Execution

Per il kernel CUDA, la scelta di `threadsPerBlock` e `numBlocks` è stata la seguente:

```
1  dim3 threadsPerBlock(16, 16);
2  dim3 numBlocks((ni + 15) / 16, (nj + 15) / 16);
3
4  // ---- CUDA events start ----
5  cudaEvent_t start, stop;
6  cudaEventCreate(&start);
7  cudaEventCreate(&stop);
8
9  // Execute the modified version using same data
10 printf("Execute GPU kernel...\n");
11 cudaEventRecord(start, 0); // start timing
12
13 for (istep = 0; istep < nstep; istep++) {
14     step_kernel_mod <<<numBlocks, threadsPerBlock>>> (ni, nj,
15         tfac, d_temp1, d_temp2);
16     cudaDeviceSynchronize();
17
18     cudaError_t err = cudaGetLastError();
19     if (err != cudaSuccess)
20         printf("CUDA Error: %s\n", cudaGetErrorString(err));
21
22     float* d_tmp = d_temp1;
23     d_temp1 = d_temp2;
24     d_temp2 = d_tmp;
25 }
26
27 cudaEventRecord(stop, 0); // stop timing
28 cudaEventSynchronize(stop);
```

Ogni blocco di 16x16 thread calcola i punti della griglia, mentre i blocchi sono organizzati per coprire l'intera matrice.

La temporizzazione è stata eseguita tramite `cudaEvent_t` per misurare il tempo del kernel GPU.

5.2.4 Results

Profiling (command: `nvprof build/cuda`)

950M:

Execute the CPU-only reference version...

- Elapsed time: 0.188 s

Execute GPU kernel...

- Elapsed time: 140.515 ms

Max Error = 0.000008

The Max Error of 0.00001 is within ACCEPTABLE bounds.

- Total elapsed time: 2.460 s

==768== Profiling application: build/cuda

==768== Profiling result:

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU:	72.95%	202.16ms	2	101.08ms	2.7665ms	199.40ms	[CUDA memcpy HtoD]
	24.93%	69.101ms	200	345.50us	342.59us	355.94us	step_kernel_mod(int, int, float, float*, float*)
	2.12%	5.8774ms	2	2.9387ms	2.5962ms	3.2812ms	[CUDA memcpy DtoH]

API:	90.99%	1.69655s	2	848.27ms	699.40us	1.69585s	cudaMalloc
	7.10%	132.39ms	200	661.97us	423.90us	7.6760ms	cudaDeviceSynchronize
	0.96%	17.932ms	4	4.4831ms	3.1292ms	7.7131ms	cudaMemcpy
	0.45%	8.4117ms	2	4.2059ms	2.4000us	8.4093ms	cudaEventCreate
	0.43%	8.0093ms	200	40.046us	9.9000us	1.0583ms	cudaLaunchKernel
	0.04%	796.50us	2	398.25us	302.30us	494.20us	cudaFree
	0.01%	213.70us	200	1.0680us	200ns	31.300us	cudaGetLastError
	0.00%	90.700us	1	90.700us	90.700us	90.700us	cudaEventSynchronize
	0.00%	81.700us	101	808ns	200ns	31.300us	cuDeviceGetAttribute
	0.00%	30.100us	2	15.050us	10.700us	19.400us	cudaEventRecord
	0.00%	5.7000us	2	2.8500us	1.3000us	4.4000us	cudaEventDestroy
	0.00%	4.3000us	1	4.3000us	4.3000us	4.3000us	cuDeviceGetPCIBusId
	0.00%	3.9000us	1	3.9000us	3.9000us	3.9000us	cudaEventElapsedTime
	0.00%	2.3000us	3	766ns	400ns	1.4000us	cuDeviceGetCount
	0.00%	1.6000us	2	800ns	300ns	1.3000us	cuDeviceGet
	0.00%	1.5000us	1	1.5000us	1.5000us	1.5000us	cuDeviceGetName
	0.00%	800ns	1	800ns	800ns	800ns	cuDeviceTotalMem
	0.00%	400ns	1	400ns	400ns	400ns	cuDeviceGetUuid

T4:

Execute the CPU-only reference version...

- Elapsed time: 36.761 s

Execute GPU kernel...

- Elapsed time: 1741.090 ms

Max Error = 0.000015

The Max Error of 0.00002 is within ACCEPTABLE bounds.

- Total elapsed time: 46.074 s

==5152== Profiling application: /content/drive/MyDrive/Colab Notebooks/HPC-CUDA/build/cuda

==5152== Profiling result:

Type	Time(%)	Time	Calls	Avg	Min	Max	Name
GPU:	67.30%	1.73103s	200	8.6551ms	8.0181ms	11.542ms	step_kernel_mod(int, int, float, float*, float*)
	16.79%	431.89ms	2	215.95ms	203.45ms	228.45ms	[CUDA memcpy HtoD]
	15.91%	409.13ms	2	204.56ms	201.12ms	208.01ms	[CUDA memcpy DtoH]
API:	62.34%	1.73679s	200	8.6840ms	8.0240ms	11.548ms	cudaDeviceSynchronize
	30.23%	842.24ms	4	210.56ms	201.46ms	228.69ms	cudaMemcpy
	7.16%	199.38ms	2	99.688ms	144.08us	199.23ms	cudaMalloc
	0.13%	3.7350ms	200	18.674us	5.4010us	429.51us	cudaLaunchKernel
	0.13%	3.6890ms	2	1.8445ms	870.86us	2.8181ms	cudaFree
	0.01%	168.47us	114	1.4770us	185ns	65.476us	cuDeviceGetAttribute
	0.00%	67.106us	200	335ns	105ns	674ns	cudaGetLastError
	0.00%	33.409us	2	16.704us	10.078us	23.331us	cudaEventRecord
	0.00%	27.898us	2	13.949us	1.1280us	26.770us	cudaEventCreate
	0.00%	12.296us	1	12.296us	12.296us	12.296us	cuDeviceGetName
	0.00%	6.8260us	1	6.8260us	6.8260us	6.8260us	cuDeviceGetPCIBusId
	0.00%	4.4120us	1	4.4120us	4.4120us	4.4120us	cudaEventSynchronize
	0.00%	3.6190us	1	3.6190us	3.6190us	3.6190us	cudaEventElapsedTime
	0.00%	3.3080us	2	1.6540us	1.1110us	2.1970us	cudaEventDestroy
	0.00%	1.6790us	3	559ns	233ns	1.2100us	cuDeviceGetCount
	0.00%	716ns	2	358ns	179ns	537ns	cuDeviceGet
	0.00%	529ns	1	529ns	529ns	529ns	cuDeviceTotalMem
	0.00%	372ns	1	372ns	372ns	372ns	cuModuleGetLoadingMode
	0.00%	302ns	1	302ns	302ns	302ns	cuDeviceGetUuid

I risultati della simulazione GPU sono stati confrontati con la versione CPU e lo speedup è calcolato come:

$$\text{speedup} = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}$$

GPU	T _{CPU} (ms)	T _{GPU} (ms)	Speedup
950M	34967	15086	x2.32
T4	37178	1645	x22.60

Table 1: Confronto tra tempi CPU e GPU e speedup

6 Conclusions

References

- [1] Simple Explanation of the Mandelbrot Set.
<https://www.wikihow.com/Plot-the-Mandelbrot-Set-By-Hand>
- [2] Mandelbrot Set – Formal Explanation.
https://en.wikipedia.org/wiki/Mandelbrot_set
- [3] Online Mandelbrot Demo.
<https://math.hws.edu/eck/js/mandelbrot/MB.html>